GLOBALPAY

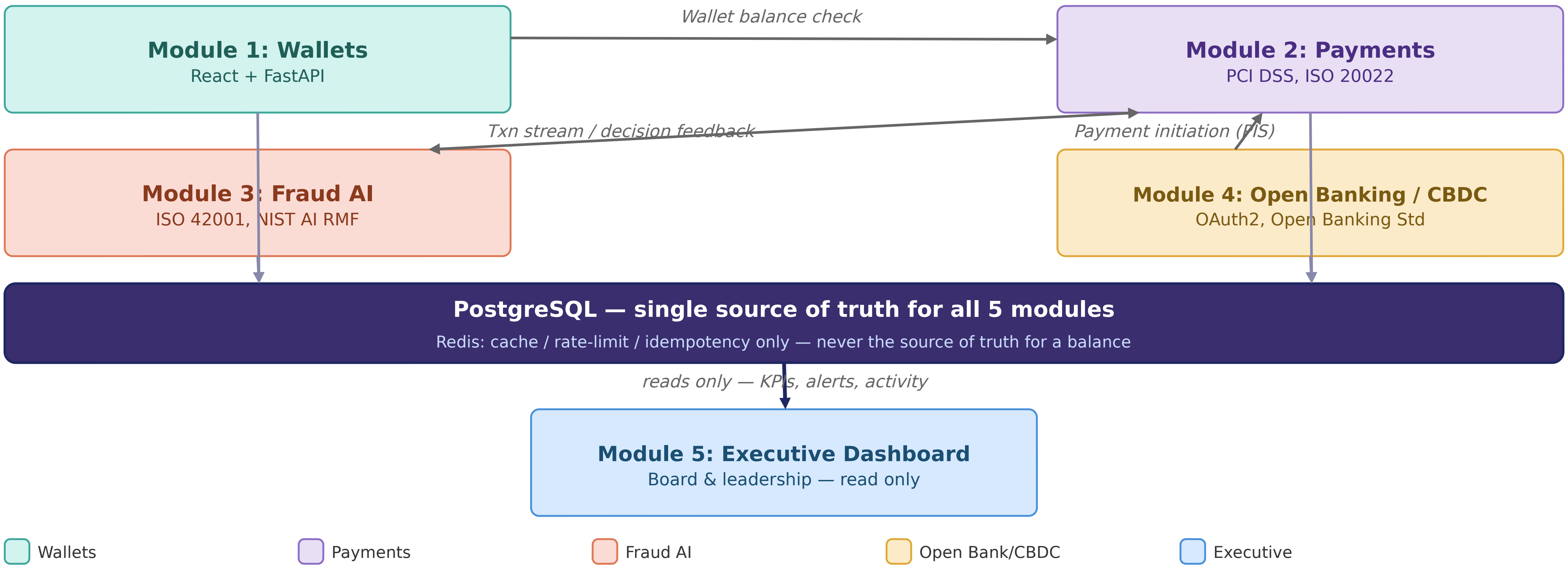
Mandatory Architecture & Engineering Deliverables

Section 8 • 17 required diagrams • numbered to match the examination brief

#	Diagram	Presentation coverage
1	Enterprise Financial Technology Architecture	Yes
2	High-Level Solution Architecture	No - here only
3	Detailed System Architecture	No - here only (composite index)
4	Digital Wallet Architecture	Yes (Module 1)
5	Payment Processing Architecture	Yes (Module 2)
6	Open Banking API Architecture	Yes (Module 4)
7	AI Fraud Detection Architecture	Yes (Module 3)
8	CBDC Simulation Architecture	No - here only
9	Enterprise Network Architecture	Yes
10	Network Flow Diagram	Yes (combined in Module 11)
11	Data Flow Diagram (Level 0)	Yes (combined)
12	Data Flow Diagram (Level 1)	Yes (combined)
13	Digital Payment Workflow	Yes (combined in Module 2)
14	Fraud Detection Workflow	Yes (combined in Module 3)
15	Open Banking API Workflow	Yes (combined in Module 4)
16	Database ERD	Yes
17	Sequence Diagram	No - here only

Enterprise Financial Technology Architecture

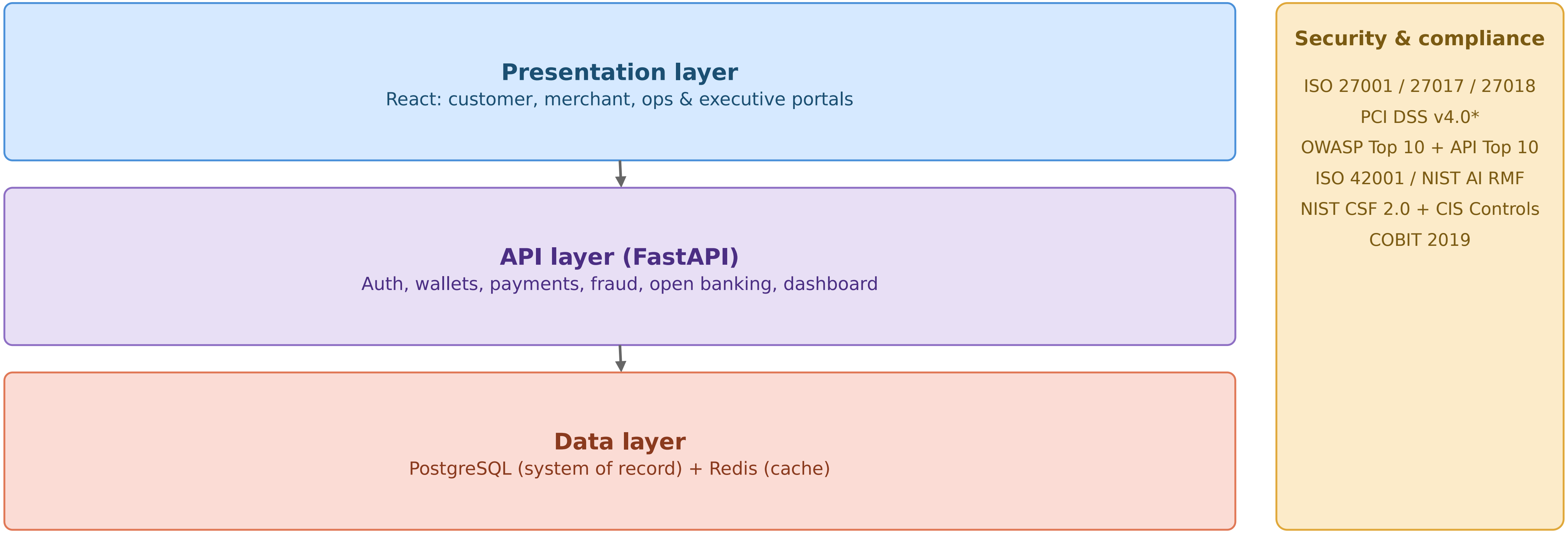
How the five modules connect



Modules 1–4 write to PostgreSQL; Module 5 reads only; only the named service calls above cross module boundaries

High-Level Solution Architecture

The 30,000-ft view



This is the 30,000-ft view — each module diagram (4-7) zooms into one API layer service

Detailed System Architecture

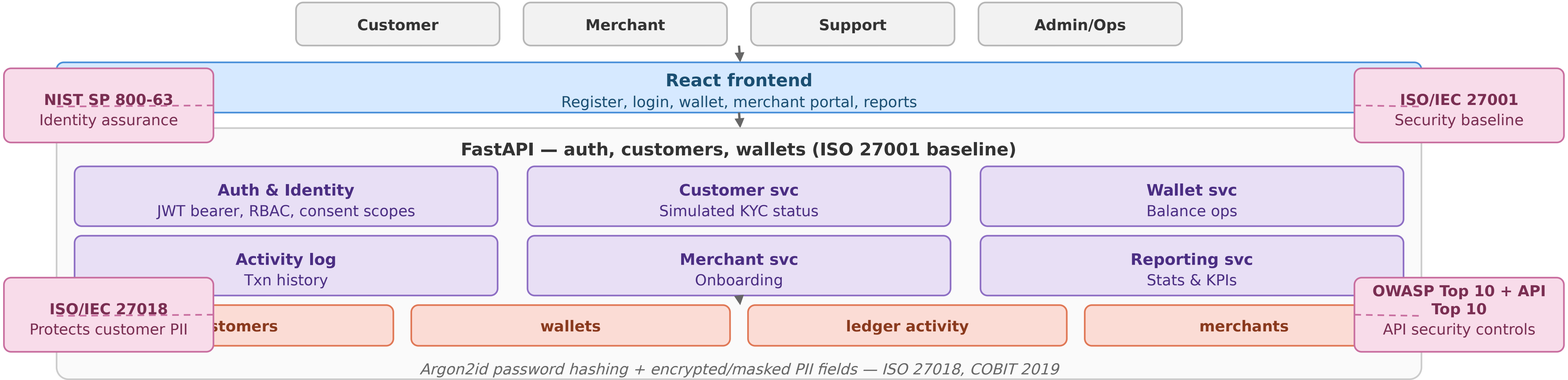
Composite reference — each module is fully detailed in diagrams 4-8

Module 1 Wallets	React UI -> FastAPI (auth, customers, wallets, merchants) -> PostgreSQL
Module 2 Payments	Validation, transaction engine, double-entry ledger -> ledger tables
Module 3 Fraud AI	Feature engineering -> Logistic Regression -> risk scoring -> human review
Module 4 Open Banking/CBDC	AIS/PIS APIs, CBDC API, activity monitoring -> separate CBDC ledger
Module 5 Executive Dashboard	Read-only aggregator across Modules 1-4 -> executive reports

This slide exists to index the detail, not duplicate it — see diagrams 4-8 for full architecture, framework badges, and data model per module

Digital Wallet Architecture

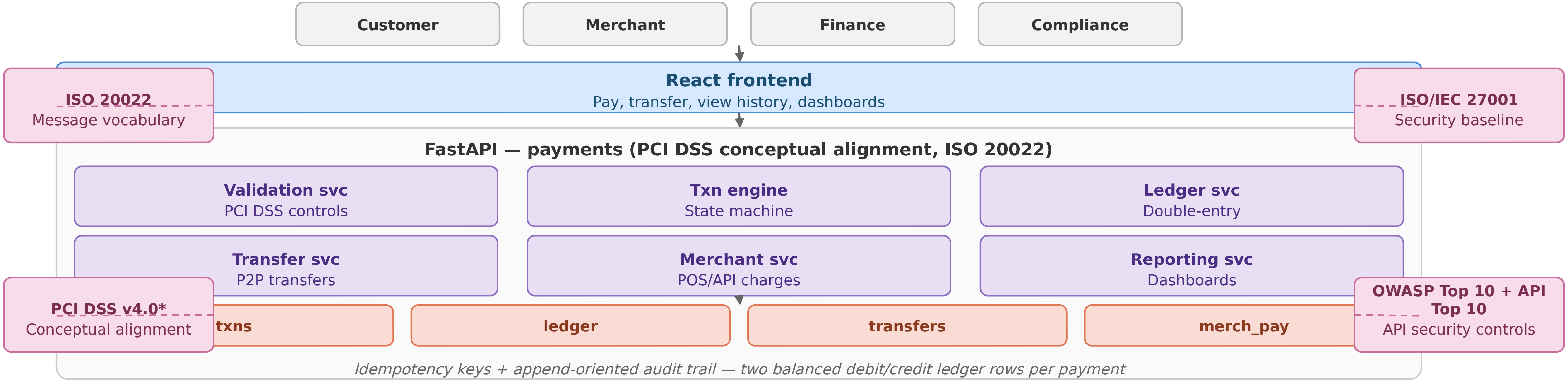
Registration, wallet creation, balance operations, merchant accounts



Writes directly into PostgreSQL — see diagram 1 for the full enterprise data-flow context

Payment Processing Architecture

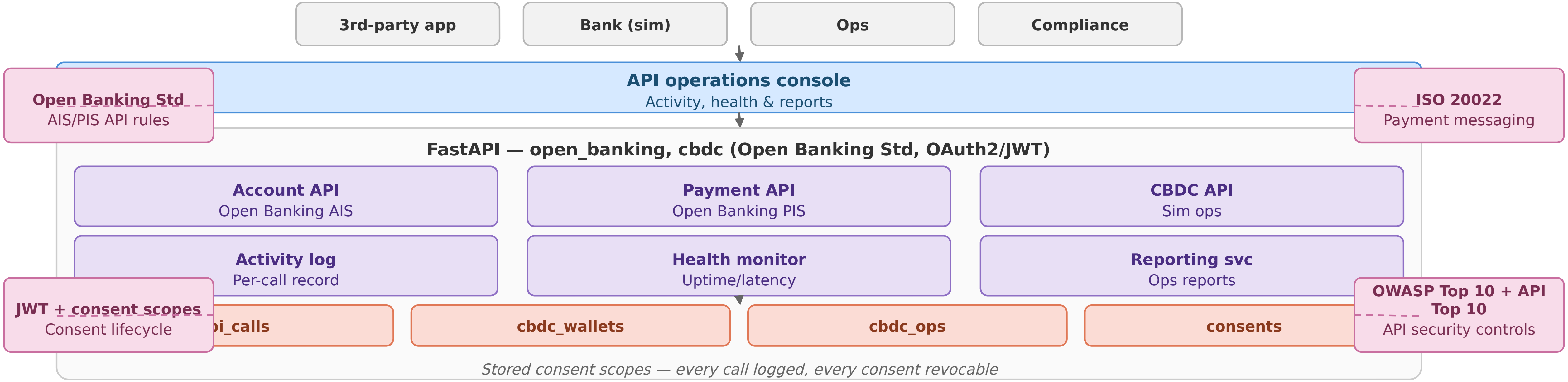
Validation, double-entry ledger, transfers, merchant charges



Direct link to Module 3 for fraud scoring — the only true module-to-module call besides Module 4's PIS

Open Banking API Architecture

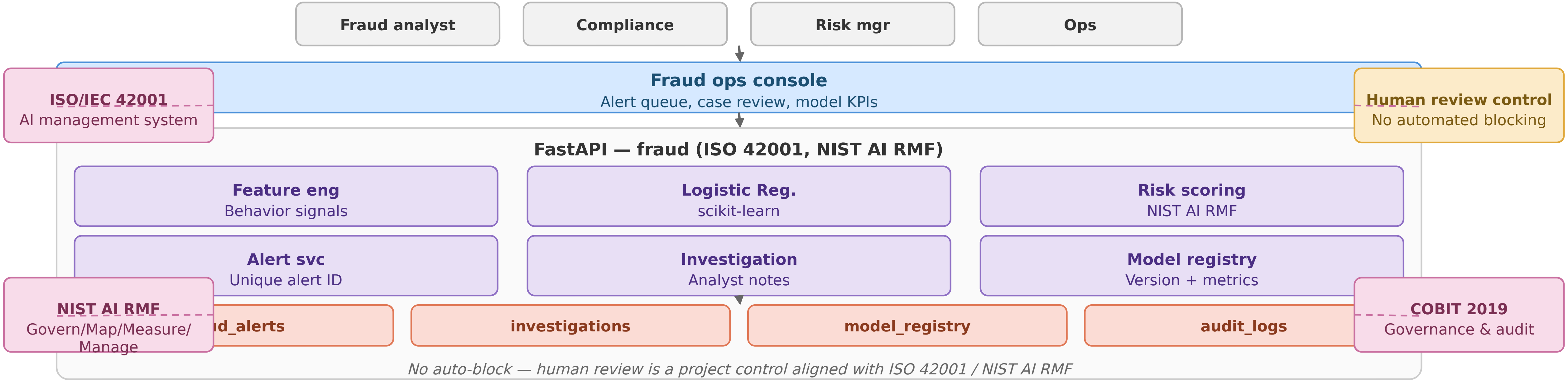
Account info & payment initiation APIs, CBDC wallet ops



PIS calls Module 2's transfer logic directly rather than duplicating it — see diagram 8 for CBDC's separate ledger

AI Fraud Detection Architecture

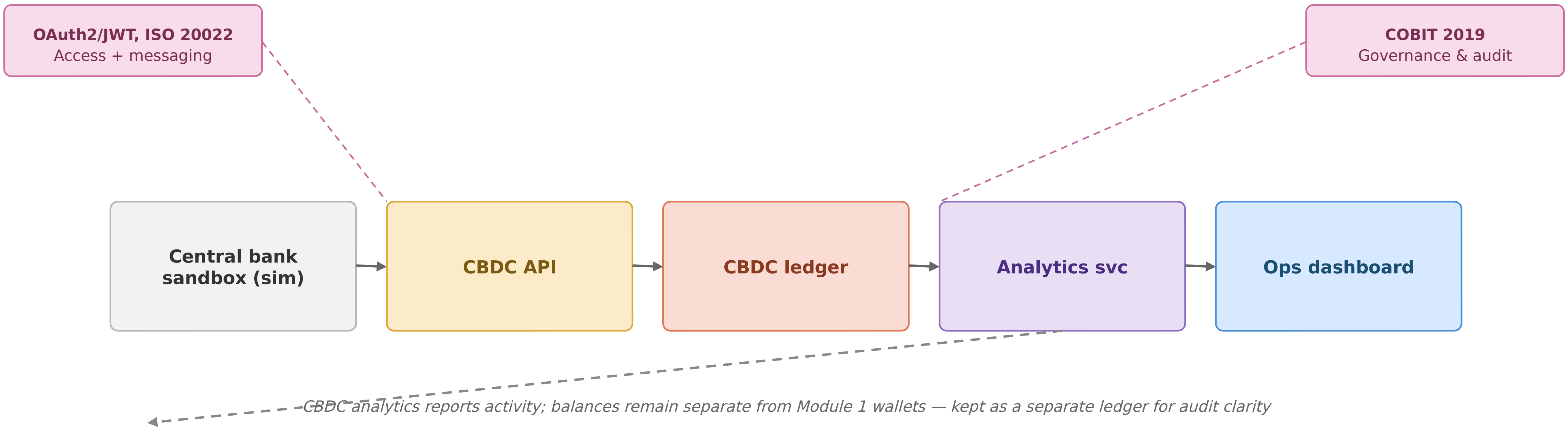
Feature engineering, classification, mandatory human review



Surfaces to fraud/risk/compliance teams only — never shown to customers

CBDC Simulation Architecture

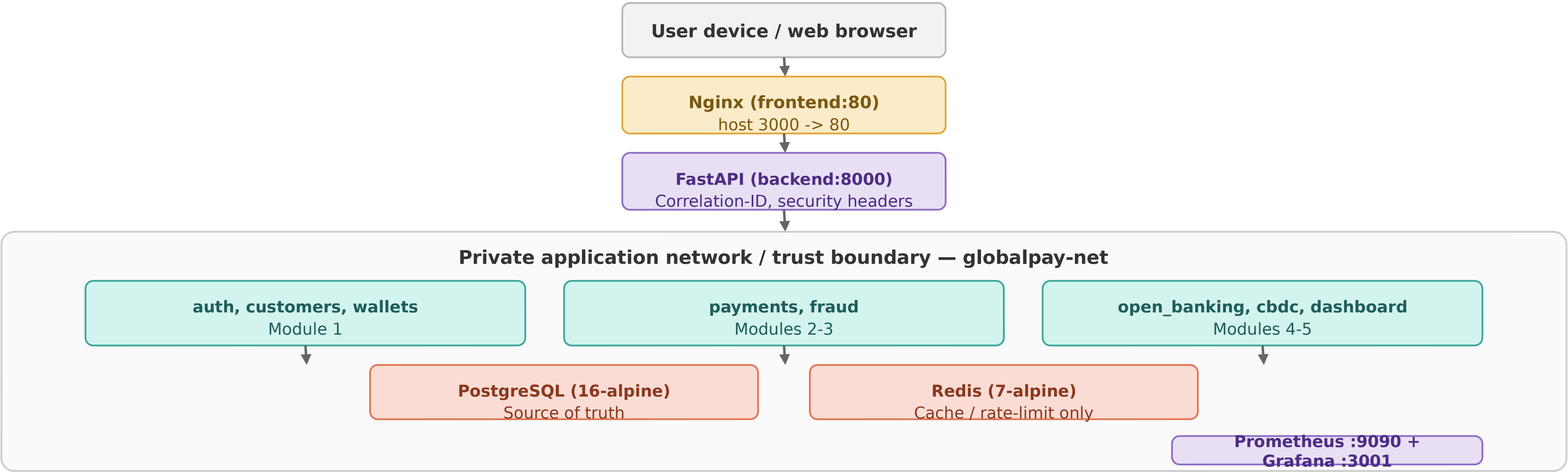
Kept separate from the commercial wallet ledger



Central-bank money is never mixed with commercial wallet balances — this table separation is the audit boundary

Enterprise Network Architecture

Host-to-container topology, trust boundaries and service dependencies



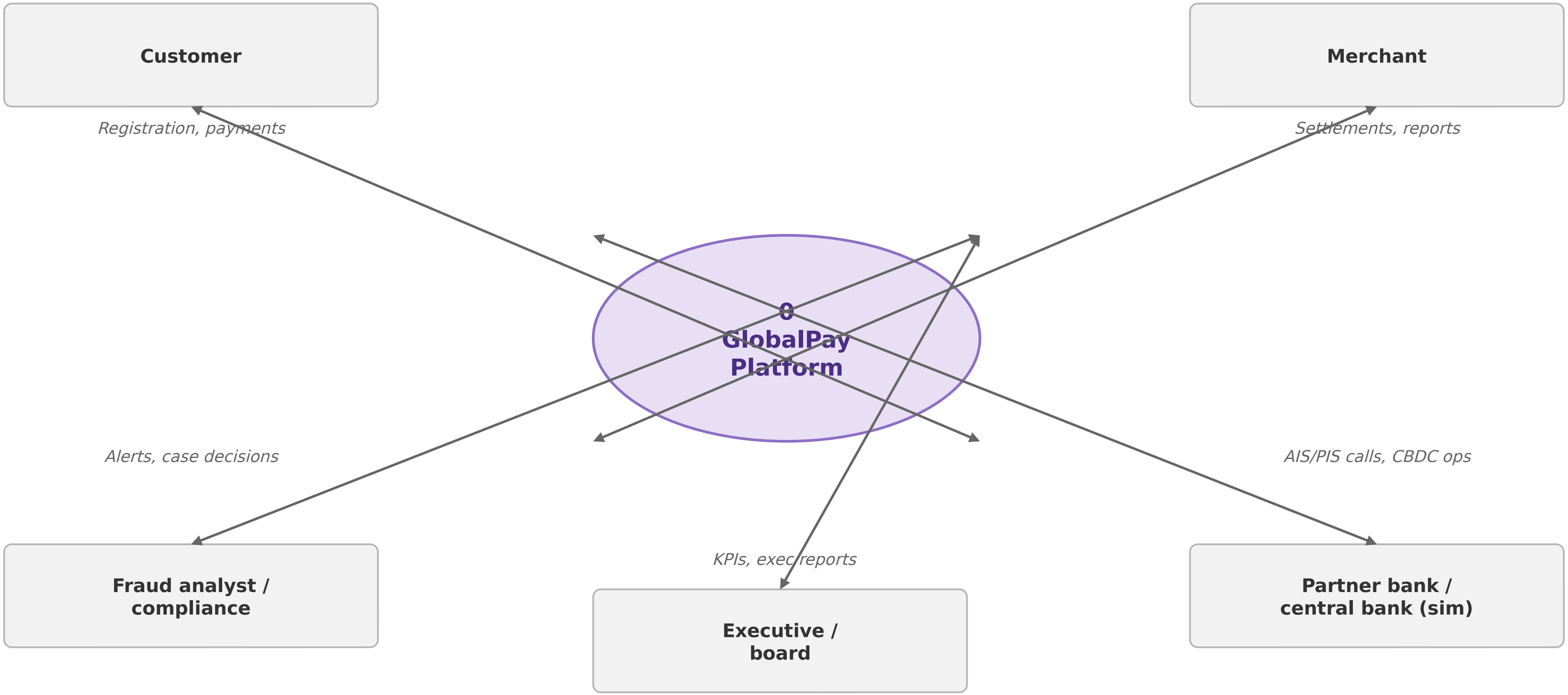
Redis never holds the authoritative wallet balance — Postgres is the single source of truth

Network Flow Diagram

How a single request traverses the network tier by tier

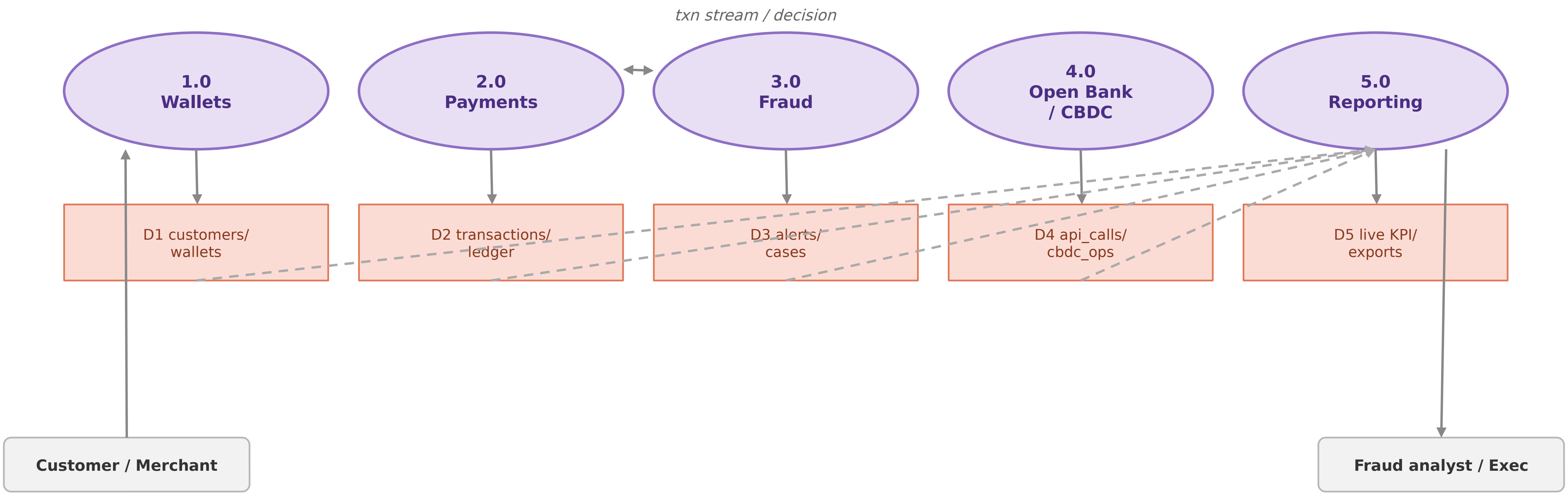
Data Flow Diagram — Level 0 (Context)

The platform as a single process, with its external entities



Data Flow Diagram — Level 1

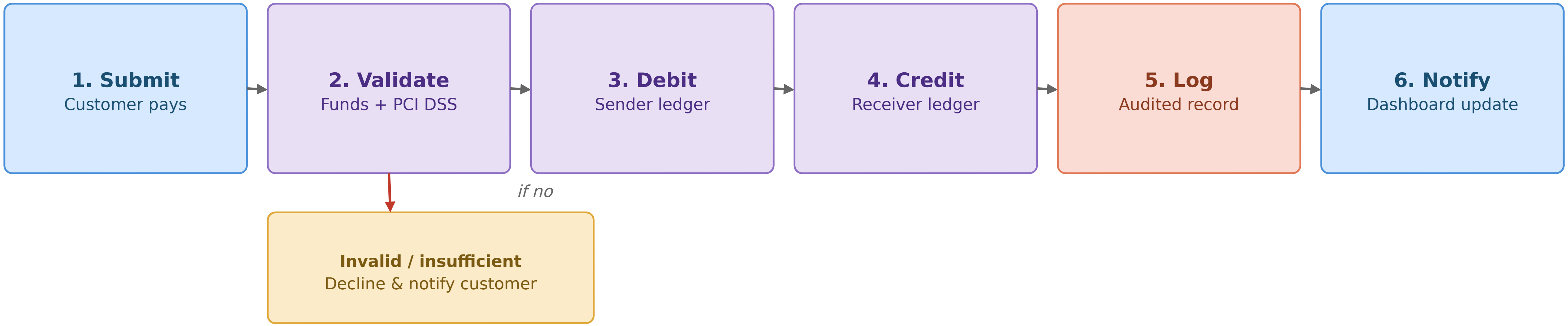
The platform decomposed into its five core processes



Dashed arrows: on-demand aggregation from data stores into 5.0 Reporting

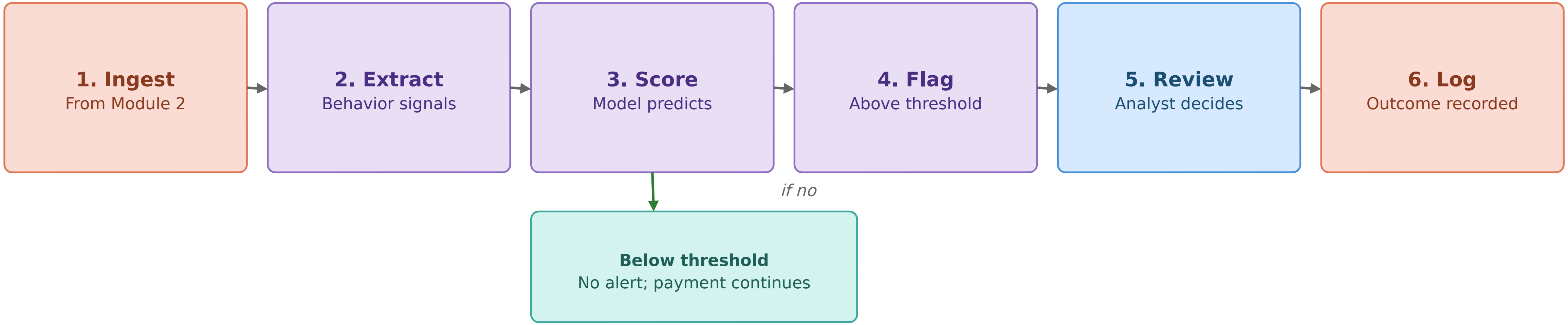
Digital Payment Workflow

Validation branch shown explicitly



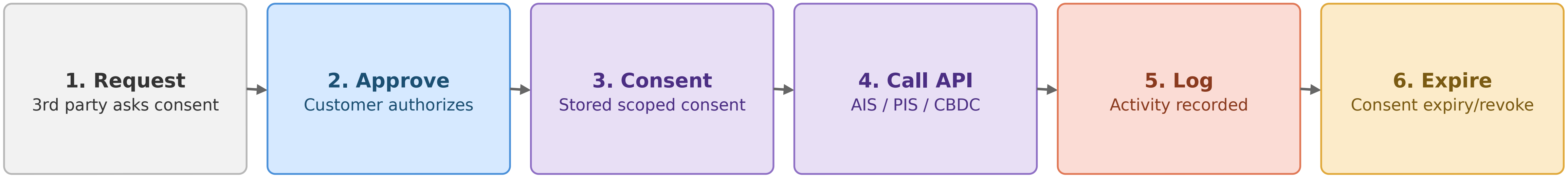
Fraud Detection Workflow

Automated scoring with mandatory human checkpoint



Open Banking API Workflow

Consent-first access to account and payment APIs



G

Database Entity Relationship Diagram (ERD)

14 tables from backend/app/models/entities.py

User
PK user_id
email, role
hashed_pw

Customer
PK customer_id
FK user_id
kyc_status

Merchant
PK merchant_id
FK user_id
business_name

Wallet
PK wallet_id
FK customer/merchant
balance

Consent
PK consent_id
FK customer_id
scopes, client

ApiCall
PK call_id
endpoint
status_code

Transaction
PK txn_id
sender/receiver_wallet
amount, status

LedgerEntry
PK entry_id
FK txn_id
direction, amount

FraudAlert
PK alert_id
FK txn_id
risk_score, status

Investigation
PK investigation_id
FK alert_id
reviewer, notes

ModelRegistry
PK model_id
version, metrics
trained_at

AuditLog
PK audit_id
action, actor
correlation_id

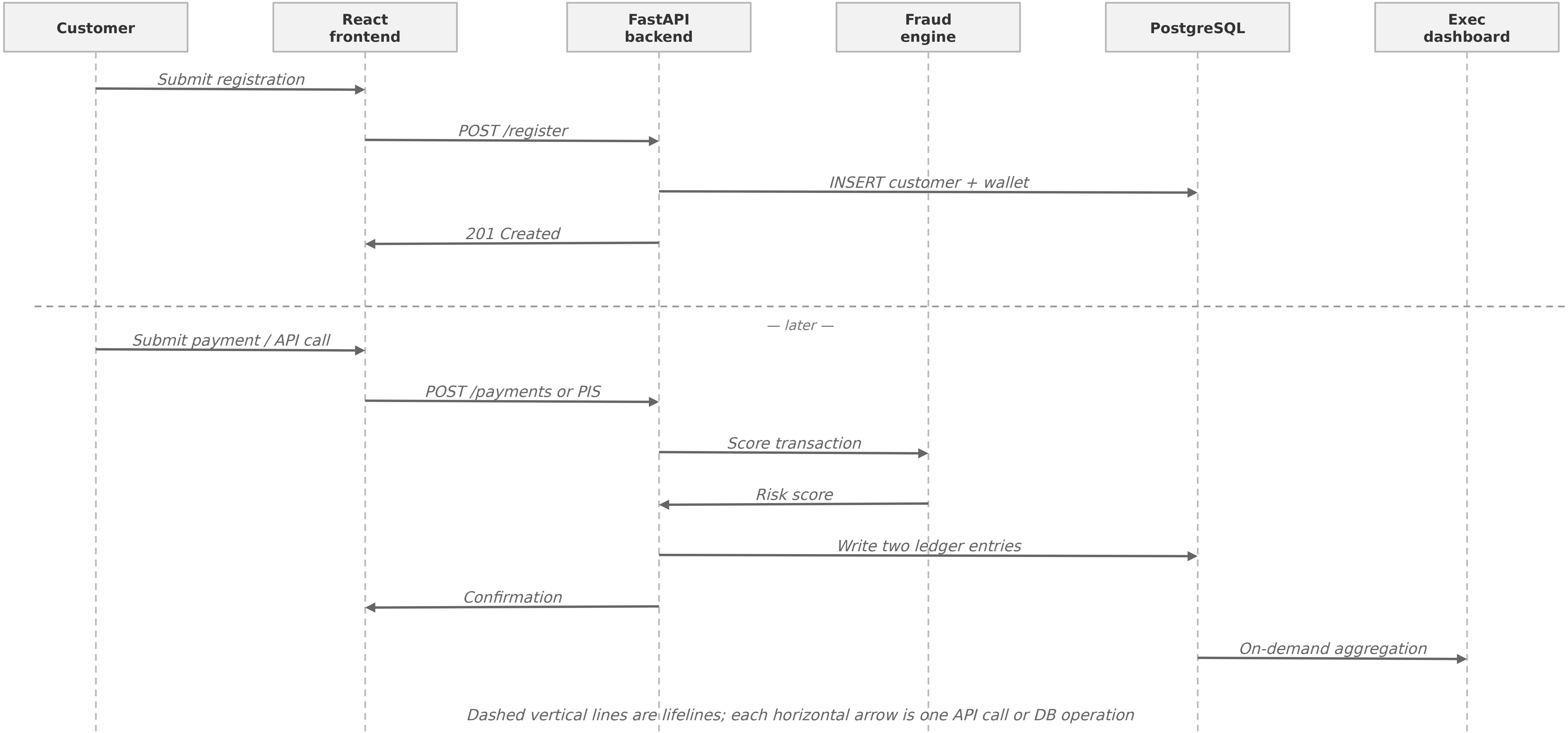
CbdcWallet
PK cbdc_wallet_id
FK customer_id
balance

CbdcOperation
PK op_id
FK cbdc_wallet_id
issue/redeem

How to read it: FK means foreign key and identifies the related table. A user may have a customer or merchant profile. Profiles own wallets. Transactions create ledger entries and may trigger fraud alerts for investigation.

Sequence Diagram — Registration Through Executive Reporting

End-to-end lifecycle across all five modules



Dashed vertical lines are lifelines; each horizontal arrow is one API call or DB operation